NIL Investment Analysis Report

Executive Summary

This report examines the relationship between projected basketball performance and estimated Name, Image, and Likeness (NIL) valuations across NCAA Division I men's basketball programs. Our objective is to develop a statistical benchmark that allows programs to compare their estimated NIL resources with teams of similar projected strength and conference affiliation.

We combined EvanMiya preseason basketball ratings with NIL estimates from The Sideline, producing a dataset of 353 teams across 32 conferences. Using multiple linear regression, we evaluated projected Bayesian Performance Rating (BPR) and conference affiliation as predictors of log-transformed NIL valuations.

The conference-adjusted model substantially outperformed the BPR-only model on the held-out test dataset. Test RMSE decreased from $1.683 million to $0.941 million, while mean squared error decreased by approximately 68.7%. These results support incorporating conference information when estimating NIL benchmarks.

We then applied the selected model to identify programs with estimated NIL valuations above or below their model-implied benchmarks, including a case study of Utah State University.

The analysis provides a framework for evaluating relative NIL valuations and identifying programs for further investigation. However, the data consist of modeled NIL estimates and projected basketball ratings, so the findings should not be interpreted as verified expenditures, causal effects, or optimal funding recommendations.

Plan

Business Objective

College Basketball programs are facing more financial pressire to allocate NIL resources effectively. Programs have to balance the financial resources available for player compensation with goal of getting a competitive roster from it. Understanding hpw NIL spending relates

to projected team performance can help programs evaluate their financial position relative to other schools in their conference.

Our goal is to estimate team NIL as a function of BPR and conference affiliation. The model will generate an NIL benchmark for each team, allowing us to compare a program's estimated NIL with the level predicted for a team with similar characteristics

Schools with estimated NIL below their benchamrk may want to do further looks as pontentially cost-efficient programs, while those above the benchmark may want to do additional review of their resource allocation

Because the analysis relies on estimated NIL values and projected rather than realized basketball performance, these comparisons are descriptive rather than causal. A difference from the model benchmark does not, by itself, establish whether a program is overpaying or underpaying its players.

Decision Problem and Objective Function

Our decision problem is to establish a reasonable NIL amount for each Division I basketball program based on its team strength and conference affiliation. These benchmarks can then support the amount of teams whose estimated NIL levels differ susbstantially from comparable programs.

Let Y_i represent the estimated NIL spending of team i, measured in millions of dollars. Let X_i contain the team's BPR and conference. The action a_i is the NIL benchmark produced by the model for that team.

We ended up using quadratic loss:

$$\ell(a_i, Y_i) = (Y_i - a_i)^2$$

Quadratic Loss penalizes larger prediction errors more heavily then smaller errors, making it appropriate for constructing benchmarks where lots of money discrepancies are big consequences

Our objective is to have a function that minimizes that expected loss

$$f^* = \arg\min_f E[(Y - f(X))^2]$$

Under squared-error loss, the best prediction for a given set of characteristics is the mean:

$$f^*(X_i) = E[Y_i \mid X_i]$$

Because we dont know the true relationship, we estimate using regression:

$$\widehat{R}(f) = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} (Y_i - \widehat{Y}_i)^2$$

After our model, we calcuate each team's estimated NIL gap:

$$G_i = \widehat{Y}_i - Y_i$$

A positive gap means that estimated NIL is below the model's benchmark, while a negative gap means estimated NIL is above the benchmark

These gaps are indicator for teams that should have further investigation rather definitive measures of financial efficiency. The available data do not establish the causal effect of NIL spending on basketball performance or determine an optimal roster budget. Furthermore, the observed outcome is a third-party NIL estimate rather than verified actual expenditure.

Here is an example:

Build

Ideal Data and Data-Generating Process

Ideally, this model would use legit verified team NIL values alongside the independently measures BPR for all Division I men's basketball programs. Things like player compensation, recruiting outcomes, and historical team performance would help improve the relationship between spending and basketball strength from other factors that influence NIL valuations

For our current objective, the variables of interest are team NIL values (estimated), Bayesian Perfomrance Rating (BPR), and conference affiliation. We expect NIL expenditure and BPR to be postively associated because stronger basketball programs tend to have access to more money and more resources, although conference affiliation and other factors may affect this relationship.

Our data we used was Sideline's estimated team NIL values and EvanMiya's team and player BPR.

Model Specification

We used a multiple linear regression model in which the natural log of team NIL is predicted by BPR and conference affiliation:

$$\log(Y_i) = \beta_0 + \beta_1 BPR_i + \sum_{c \neq \text{BigTen}} \gamma_c I(C_i = c) + \epsilon_i$$

where:

$$\epsilon_i \overset{iid}{\sim} N(0, \sigma^2)$$

NIL was log-transformaed becuase NIL estimates have a very right skew. This transformation allows the relationship between BPR and NIl to be seen on a log scale.

The coefficient β_1 represents the expected change in log NIL associated with a +1 increase in BPR, holding all other variables constant. Its exact interpretation is $100(e^{\beta_1} - 1)$.

Conference was represented using indicator variables, with the Big Ten as the reference conference. These indicators allow teams from different conferences to have different intercepts while assuming a common BPR slope

The initial model assumes linearity on the log scale, independent errors, constant error variance, and normally distributed errors. It also assumes that the effect of BPR does not vary by conference. These simplifying assumptions will be evaluated using the observed data.

Under these distributional assumptions, the model-implied conditional mean NIL on the original dollar scale is:

$$E[Y_i \mid X_i] = e^{\mu_i + \sigma^2/2}$$

where μ_i is the predicted log NIL. This distinction matters because simply exponentiating μ_i produces the conditional median under the log-normal model, rather than its conditional mean.

Simulating Data and Recovering Parameters

```python
import numpy as np
import polars as pl
import statsmodels.formula.api as smf
from plotnine import *

# Known parameters for the simulated data
beta_0 = 1.40
beta_bpr = 0.030
sigma = 0.30

conference_effects = {
    'BigTen': 0.00,
    'BigEast': -0.25,
    'WCC': -0.65,
    'Summit': -1.00
}

# Function to simulate team NIL data
def simulate_nil(n, beta_0, beta_bpr, conference_effects, sigma):
    rng = np.random.default_rng(42)

    # Assign conferences and generate correlated BPR values
    conference = rng.choice(
        ['BigTen', 'BigEast', 'WCC', 'Summit'],
        size=n,
        p=[0.20, 0.20, 0.25, 0.35]
    )

    bpr_means = {
        'BigTen': 12,
        'BigEast': 7,
        'WCC': 0,
        'Summit': -7
    }

    bpr = (
        np.array([bpr_means[c] for c in conference])
        + rng.normal(0, 6, size=n)
    )

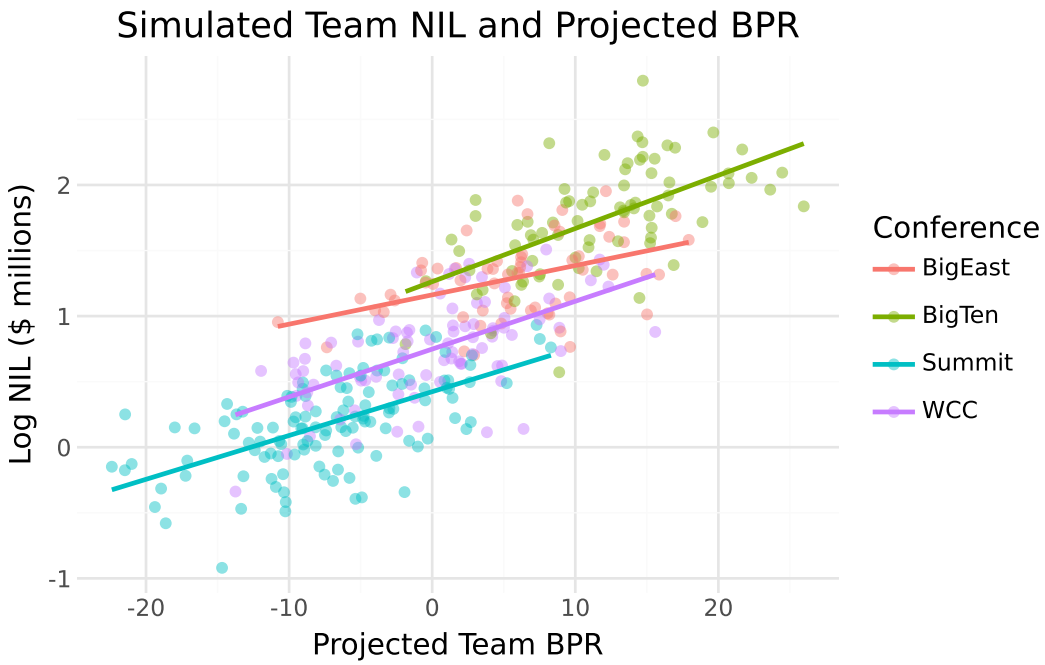
    # Generate normally distributed errors
    error = rng.normal(0, sigma, size=n)
```

```
    # Generate log NIL using the specified linear model
    log_nil = (
        beta_0
        + beta_bpr * bpr
        + np.array([conference_effects[c] for c in conference])
        + error
    )

    return pl.DataFrame({
        'conference': conference,
        'bpr': bpr,
        'log_nil': log_nil,
        'nil_m': np.exp(log_nil)
    })

# Simulate 353 teams
sm_data = simulate_nil(
    n=353,
    beta_0=beta_0,
    beta_bpr=beta_bpr,
    conference_effects=conference_effects,
    sigma=sigma
)
```

```
# Visualize the simulated relationship by conference
(ggplot(sm_data, aes(x='bpr', y='log_nil', color='conference'))
    + geom_point(alpha=0.45)
    + geom_smooth(method='lm', se=False)
    + labs(
        title='Simulated Team NIL and Projected BPR',
        x='Projected Team BPR',
        y='Log NIL ($ millions)',
        color='Conference'
    )
    + theme_minimal()
)
```




```
# Dummy code conference, using Big Ten as the reference
sm_train = (
    sm_data
    .to_dummies(columns=['conference'], drop_first=False)
    .select(pl.exclude('conference_BigTen'))
)

# Fit the same linear relationship used to generate the data
sim_fit = smf.ols(
    'log_nil ~ bpr + conference_BigEast + conference_WCC + conference_Summit',
    data=sm_train
).fit()
```

```
# Store the parameters used to generate the simulated data
true_parameters = {
    'Intercept': beta_0,
    'bpr': beta_bpr,
    'conference_BigEast': conference_effects['BigEast'],
    'conference_WCC': conference_effects['WCC'],
    'conference_Summit': conference_effects['Summit']
}

# Extract estimated coefficients and confidence intervals
```

```
sim_ci = sim_fit.conf_int()

recovery = pl.DataFrame({
    'parameter': list(true_parameters),
    'true_value': list(true_parameters.values()),
    'estimate': [
        sim_fit.params[p] for p in true_parameters
    ],
    'lower_95': [
        sim_ci.loc[p, 0] for p in true_parameters
    ],
    'upper_95': [
        sim_ci.loc[p, 1] for p in true_parameters
    ]
})

# Check whether the intervals contain the true parameters
recovery = recovery.with_columns(
    contains_true=pl.col('true_value').is_between(
        pl.col('lower_95'),
        pl.col('upper_95')
    )
)

recovery.with_columns(pl.col(pl.Float64).round(3))
```

parameter str	true_value f64	estimate f64	lower_95 f64	upper_95 f64	contains_true bool
”Intercept”	1.4	1.341	1.252	1.43	true
”bpr”	0.03	0.034	0.029	0.039	true
”conference_BigEast”	-0.25	-0.246	-0.349	-0.143	true
”conference_WCC”	-0.65	-0.593	-0.701	-0.484	true
”conference_Summit”	-1.0	-0.914	-1.041	-0.787	true

This simulation was done to look at whether OLS could recover known parameters under the data-generating process. We did 353 observations using the BPR coefficient, conference-specfice intercepts, and normally distributed error.

The OLS model recovered parameter estimates pretty close to the values used to generate the data. The estimated BPR coefficient was about 0.034 compared with the actual value of 0.30.

The estimated conference effectes were also consistent with the relationships, and the 95% confidence intervals has the true parameters in this simulation

These results show that the estimations can recover the relationships when we assumed linear model correctly represents the data process. However, successful parameter recovery in simulated data does not establish that the same assumptions are valid for the observed NIL data.

Explore

Data Sources and Collection

This analysis uses two datasets:

The first dataset is from EvanMiya

The second data set is from The Sideline Men's Basketball NIL Tracker

Both datasets are downloaded as CSV files and were joined using school names. Since the tow sources has different names for some schools, they were standardized before merging.

Data Cleaning and Preparation

Load datasets

```
from pyprojroot.here import here

# Load the original 2026-27 team ratings and NIL estimates
team_data = pl.read_csv(
    here('data/evanmiya_team_ratings_2026_27.csv')
)

nil_data = pl.read_csv(
    here('data/sideline-basketball-nil-rankings.csv'),
    skip_rows=3
)
```

Standardize school names

```
# Standardize Sideline school names to match EvanMiya
school_names = {
  'American University': 'American', 'Detroit Mercy': 'Detroit', 'FIU': 'Florida Internationa
  'Grambling State': 'Grambling', 'IU Indianapolis': 'IU Indy', 'LIU': 'Long Island',
  'Louisiana': 'Louisiana-Lafayette', 'Louisiana Monroe': 'Louisiana-Monroe',
  'Loyola (Maryland)': 'Loyola Maryland', 'Maryland Eastern Shore': 'Maryland-Eastern Shore'
  'McNeese': 'McNeese State', 'Miami (FL)': 'Miami (Fla.)', 'NC A&T': 'North Carolina A&T',
  'NC Central': 'North Carolina Central', 'Nicholls': 'Nicholls State', 'Prairie View A&M':
  'Purdue Fort Wayne': 'Fort Wayne', 'Queens University': 'Queens', 'SE Missouri State': 'Sou
  'SIUE': 'SIU Edwardsville', 'Sam Houston': 'Sam Houston State', 'Seattle University': 'Seat
  'Southern Miss': 'Southern Mississippi', 'St. Thomas': 'St. Thomas (MN)', 'UConn': 'Connect
  'UIC': 'Illinois-Chicago', 'USC Upstate': 'South Carolina Upstate', 'UT Martin': 'Tennessee
  'UTRGV': 'Texas-Rio Grande Valley'
}
tier_names = {'Power 4': 'Power4', 'Lower-Tier': 'LowerTier'}


# Clean Sideline data and include conference
nil_clean = (
    nil_data
    .select(
        pl.col('School').alias('team'),
        pl.col('Tier').alias('tier'),
        pl.col('Conference').alias('conference'),
        pl.col("Men's Basketball NIL Estimate ($M)").alias('nil_m')
    )
    .with_columns(
        pl.col('team').replace(school_names),
        pl.col('tier').replace(tier_names),
        pl.col('conference').str.replace_all('[^A-Za-z0-9]', ''),
        pl.col('nil_m').cast(pl.Float64, strict=False)
    )
    .unique()
)


# Keep only the useful EvanMiya columns
team_clean = (team_data
  .select('team', 'rank', 'obpr', 'dbpr', 'bpr', 'tempo')
  .unique()
)
```

Join datasets

```
# Join EvanMiya ratings with Sideline NIL estimates
tm_data = (
    team_clean
    .join(nil_clean, on='team', how='inner', validate='1:1')
    .with_columns(
        pl.col('nil_m').log().alias('log_nil')
    )
    .sort('team')
)

# Verify the final columns
print(tm_data.columns)
print("Matched teams:", tm_data.height)
```

```
['team', 'rank', 'obpr', 'dbpr', 'bpr', 'tempo', 'tier', 'conference', 'nil_m', 'log_nil']
Matched teams: 353
```

Descriptive Statistics

Check missing data

```
# Teams in EvanMiya without a matching Sideline record
dropped = team_clean.join(
    nil_clean.select('team'),
    on='team',
    how='anti'
)

# Sideline teams without a matching EvanMiya record
unmatched_nil = nil_clean.join(
    team_clean.select('team'),
    on='team',
    how='anti'
)

# Summarize matching coverage
coverage = pl.DataFrame({
```

```
    'source': [
        'EvanMiya teams',
        'Sideline teams',
        'Matched teams',
        'Unmatched EvanMiya',
        'Unmatched Sideline'
    ],
    'teams': [
        team_clean.height,
        nil_clean.height,
        tm_data.height,
        dropped.height,
        unmatched_nil.height
    ]
})

print(coverage)


print("Missing values in the matched data:")
print(tm_data.null_count())
```

```
shape: (5, 2)

 source               teams
 ---                  ---
 str                  i64

 EvanMiya teams       365
 Sideline teams       354
 Matched teams        353
 Unmatched EvanMiya   12
 Unmatched Sideline   1

Missing values in the matched data:
shape: (1, 10)

 team   rank   obpr   dbpr   …   tier   conference   nil_m   log_nil
 ---    ---    ---    ---        ---    ---          ---     ---
 u32    u32    u32    u32        u32    u32          u32     u32

 0      0      0      0      …   0      0            0       0
```

Overall Descriptive Statistics

```
# Describe the outcome, primary predictor, and sample
overall_summary = tm_data.select(
    pl.len().alias('teams'),
    pl.col('conference').n_unique().alias('conferences'),
    pl.col('nil_m').mean().alias('mean_nil_m'),
    pl.col('nil_m').median().alias('median_nil_m'),
    pl.col('nil_m').min().alias('min_nil_m'),
    pl.col('nil_m').max().alias('max_nil_m'),
    pl.col('bpr').mean().alias('mean_bpr'),
    pl.col('bpr').std().alias('sd_bpr'),
    pl.corr('bpr', 'log_nil').alias('cor_bpr_log_nil')
)

overall_summary.with_columns(
    pl.col(pl.Float64).round(3)
)
```

teams u32	conferences u32	mean_nil_m f64	median_nil_m f64	min_nil_m f64	max_nil_m f64	mean_bpr f64	sd_bpr f64	cor_ f64
353	32	2.8	1.15	0.3	15.8	0.289	10.379	0.81

By tier

```
# Summarize NIL and projected team strength by tier
tier_summary = (
    tm_data
    .group_by('tier')
    .agg(
        teams=pl.len(),
        mean_nil_m=pl.col('nil_m').mean(),
        median_nil_m=pl.col('nil_m').median(),
        mean_bpr=pl.col('bpr').mean(),
        sd_bpr=pl.col('bpr').std(),
        correlation=pl.corr('bpr', 'log_nil')
```

```
    )
    .sort('median_nil_m', descending=True)
)

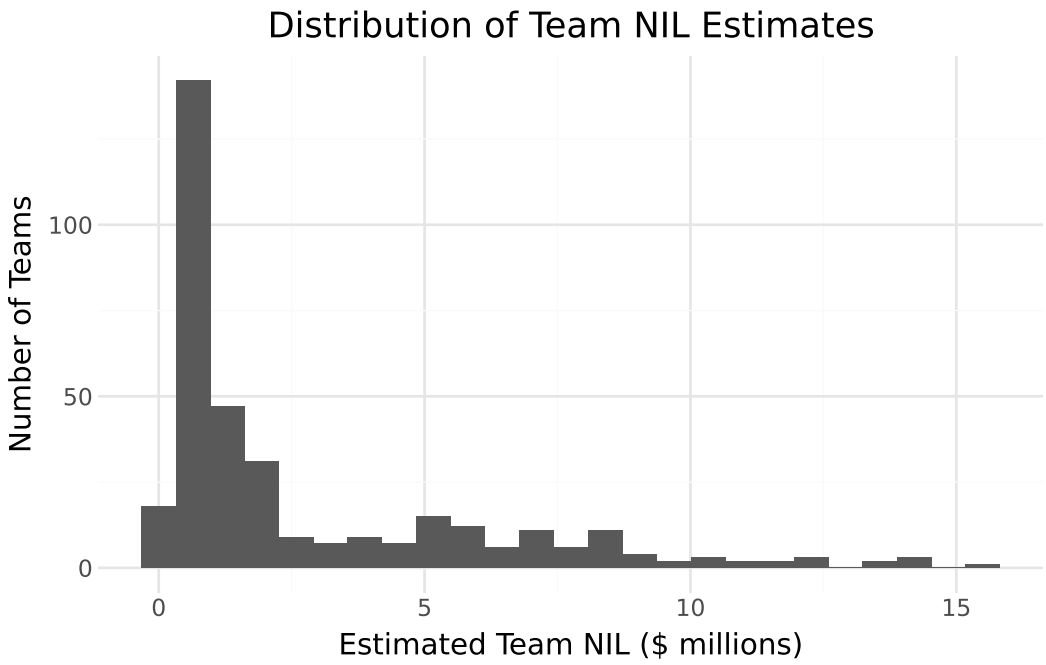
tier_summary.with_columns(
    pl.col(pl.Float64).round(3)
)
```

tier str	teams u32	mean_nil_m f64	median_nil_m f64	mean_bpr f64	sd_bpr f64	correlation f64
”Power4”	68	7.526	6.925	15.282	6.612	0.619
”G5”	53	2.362	1.9	0.674	7.009	0.56
”Basketball”	56	3.266	1.6	0.761	9.476	0.786
”LowerTier”	100	1.104	0.825	-4.714	5.177	0.552
”FCS”	68	0.807	0.75	-7.737	6.073	0.117
”Ivy”	8	0.413	0.4	-2.248	4.299	0.668

Exploratory Data Analysis

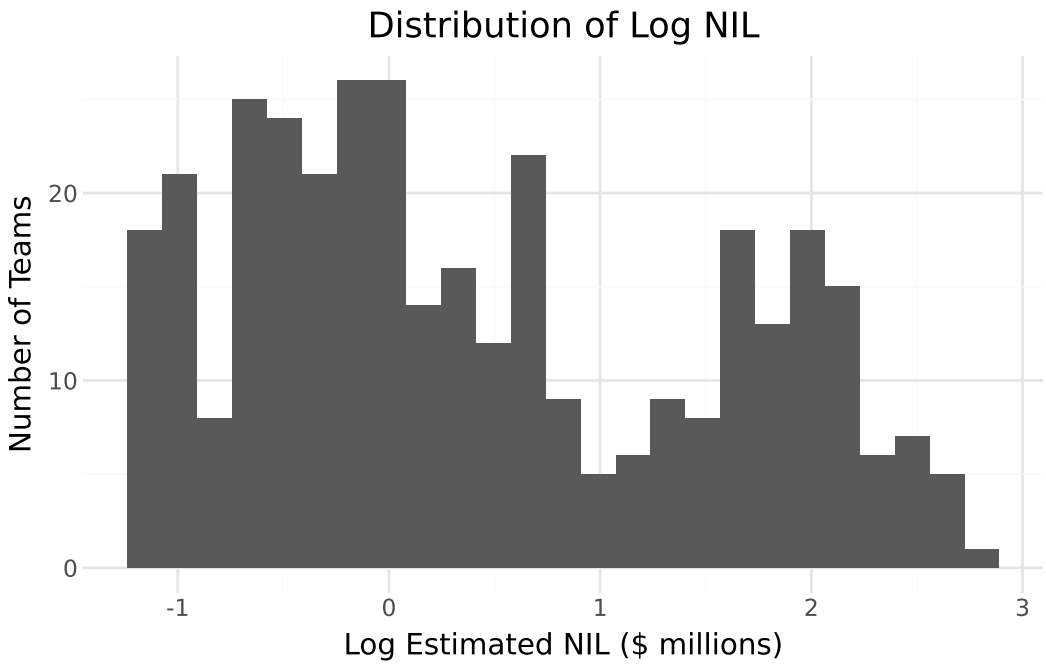
Viz 1: Estimated NIL distribution

```
(ggplot(tm_data, aes(x='nil_m'))
    + geom_histogram(bins=25)
    + labs(
        title='Distribution of Team NIL Estimates',
        x='Estimated Team NIL ($ millions)',
        y='Number of Teams'
    )
    + theme_minimal()
)
```



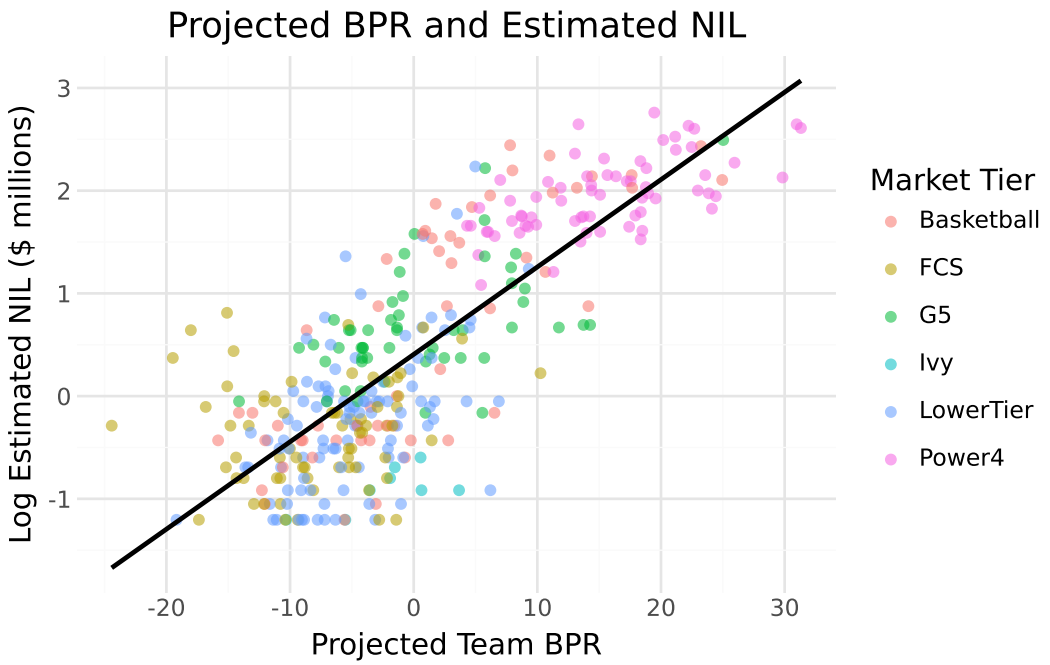
Viz 2: Log NIL

```
(ggplot(tm_data, aes(x='log_nil'))
    + geom_histogram(bins=25)
    + labs(
        title='Distribution of Log NIL',
        x='Log Estimated NIL ($ millions)',
        y='Number of Teams'
    )
    + theme_minimal()
)
```



Viz 3: BPR and Log NIL

```
(ggplot(tm_data, aes(x='bpr', y='log_nil'))
    + geom_point(aes(color='tier'), alpha=0.55)
    + geom_smooth(
        method='lm',
        se=False,
        color='black'
    )
    + labs(
        title='Projected BPR and Estimated NIL',
        x='Projected Team BPR',
        y='Log Estimated NIL ($ millions)',
        color='Market Tier'
    )
    + theme_minimal()
)
```



Data Limitations

We have a couple of limitations that affect the interpretation and gereralizability of our data

First, some of our NIL estimates from Sideline are a modeled values rather than the verified compensation. So our outcome should be interpreted as estimated NIL market and not the actual audit

Second, the analysis uses cross-sectional data froma single season. It doesn't capture how changes in NIL spending affect performance over time, and it doesn't account for other facotrs like program history, popularity, recruiting, donors, or how a player has a specific relationship with a college

Reconcile

Following our workflow, we start with a linear model predicting log NIL from BPR and market. We then look into whether conference give a better representation of differences between schools.

Our model diagnostics are conducted using our training data, while the held-out test data are kept for the predictive model evaluation. This divide reduces the risk of choosing a model based on its performance on data just to evaluate it

Validity and Representativeness

Validity

The validity of our analysis depends on whether our variables represent our objective. Our objective is to build NIL benchmarks associated with basketball strength and conference.

BPR gives us a quantitative measure of team performance, and Sideline's estimated NIL values give us a reference for financial resources. Both variables are relevant to our objective, although neither directly measures realized performance or verfied NIL value

Representativeness

We have 353 of the 365 teams from our datasets which 96.7% coverage.

The 12 unmatched programs have an average projected BPR of -8.539, compared with 0.289 among matched programs. This difference suggests that lower-performing teams are underrepresented in the analytical sample.

Our data will be examines using DFFITS. Big influence stats can identify teams that substantially affect fitted values, but those teams will not be removed just because they are unusual.

Linearity

80/20 Split

```
from sklearn.model_selection import train_test_split
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm

# Sort teams for reproducibility
model_data = tm_data.sort('team')

# Reserve 20% of observations for final model evaluation
train_raw, test_raw = train_test_split(
    model_data,
    test_size=0.20,
    random_state=42
)
```

```
print("Training observations:", train_raw.height)
print("Testing observations:", test_raw.height)
print("Training conferences:", train_raw['conference'].n_unique())

# Verify both sets contain different teams
assert not (
    set(train_raw['team'].to_list())
    & set(test_raw['team'].to_list())
)
```

```
Training observations: 282
Testing observations: 71
Training conferences: 32
```

Fit the BPR and Tier model

```
# Create tier indicator variables (Power4 is the reference)
tier_train = (
    train_raw
    .to_dummies(columns=['tier'], drop_first=False)
    .select(pl.exclude('tier_Power4'))
)

tier_cols = sorted(
    c for c in tier_train.columns
    if c.startswith('tier_')
)

# Estimate log NIL from BPR and market tier
fit_tier = smf.ols(
    'log_nil ~ bpr + ' + ' + '.join(tier_cols),
    data=tier_train.to_pandas()
).fit()

# Attach residuals and fitted values in their original row order
tier_diag = tier_train.with_columns(
    pl.Series('resid_tier', fit_tier.resid.to_numpy()),
    pl.Series('fitted_tier', fit_tier.fittedvalues.to_numpy())
)
```

```
print("Initial R-squared:", round(fit_tier.rsquared, 3))
print("Initial adjusted R-squared:", round(fit_tier.rsquared_adj, 3))
print("BPR coefficient:", round(fit_tier.params['bpr'], 4))
```

```
Initial R-squared: 0.729
Initial adjusted R-squared: 0.723
BPR coefficient: 0.0551
```

Independence

See the conference residuals

```
# Check whether conferences are nested within tiers
tier_nesting = (
    train_raw
    .group_by('conference')
    .agg(n_tiers=pl.col('tier').n_unique())
)

print(
    "Maximum tiers per conference:",
    tier_nesting['n_tiers'].max()
)

# Summarize residuals from the initial tier model by conference
conf_resid_initial = (
    tier_diag
    .group_by('conference')
    .agg(
        teams=pl.len(),
        mean_residual=pl.col('resid_tier').mean()
    )
    .sort('mean_residual')
)

# Display conferences with the largest negative and positive means
with pl.Config(tbl_rows=12):
    print(
        pl.concat([
            conf_resid_initial.head(5),
```

```
            conf_resid_initial.tail(5)
        ])
    )
```

```
Maximum tiers per conference: 1
shape: (10, 3)

 conference       teams   mean_residual
 ---              ---     ---
 str              u32     f64

 Patriot          3       -0.955064
 Southland        9       -0.632489
 MAAC             12      -0.499504
 MissouriValley   8       -0.448286
 WAC              7       -0.384914
 ConferenceUSA    10      0.33496
 BigEast          9       0.563489
 Atlantic10       13      0.56724
 WestCoast        9       0.87056
 MEAC             4       1.095628
```

Do model again with conference instead of tier

```
# Replace market tier with conference indicators
# Big Ten is the reference conference
cf_train = (
    train_raw
    .to_dummies(columns=['conference'], drop_first=False)
    .select(pl.exclude('conference_BigTen'))
)

conf_cols = sorted(
    c for c in cf_train.columns
    if c.startswith('conference_')
)

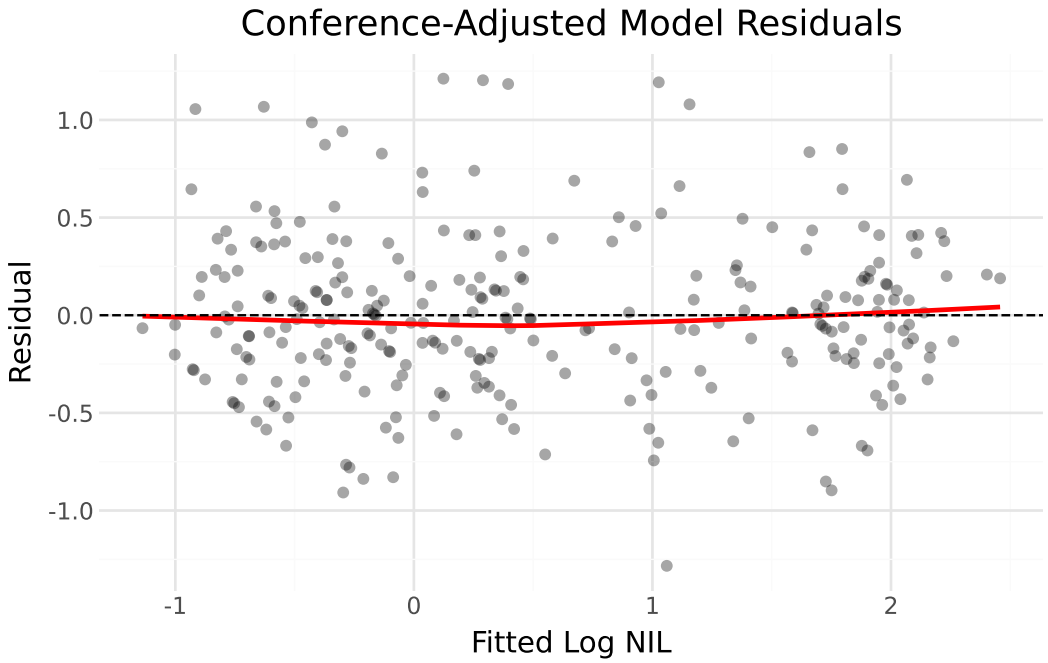
assert 'conference_BigTen' not in conf_cols
```

```
# Fit conference-adjusted OLS
fit_conf = smf.ols(
    'log_nil ~ bpr + ' + ' + '.join(conf_cols),
    data=cf_train.to_pandas()
).fit()


# Save residuals and fitted values, preserving conference names
cf_diag = (
    cf_train
    .with_columns(
        pl.Series('resid_conf', fit_conf.resid.to_numpy()),
        pl.Series('fitted_conf', fit_conf.fittedvalues.to_numpy())
    )
    .join(
        train_raw.select('team', 'conference'),
        on='team',
        how='left',
        validate='1:1'
    )
)
```

Check model again rq

```
(ggplot(cf_diag, aes(x='fitted_conf', y='resid_conf'))
    + geom_point(alpha=0.35)
    + geom_smooth(method='lowess', se=False, color='red')
    + geom_hline(yintercept=0, linetype='dashed')
    + labs(
        title='Conference-Adjusted Model Residuals',
        x='Fitted Log NIL',
        y='Residual'
    )
    + theme_minimal()
)
```



The inital specfication models log NIL using BPR and market tier as the indicators. We then looked at conference residuals to see whether variation stayed within the market classifications

Because each conference belongs to a tier in the data, tier and conference indicators overlap information. That would be perfect multicollinearity.

So we replaced tier with conference. The new model has conference-specific intercepts while keeping a common BPR slope, with the Big Ten as our reference group

The adjustment is supposed to account for the fundamental difference in epxected NIL between conferences.

Constant Variance

Breusch-Pagan Test

```
# Test for nonconstant residual variance
bp_lm, bp_pvalue, bp_f, bp_f_pvalue = het_breuschpagan(
    fit_conf.resid,
    fit_conf.model.exog
)

print("Breusch-Pagan LM statistic:", round(bp_lm, 3))
```

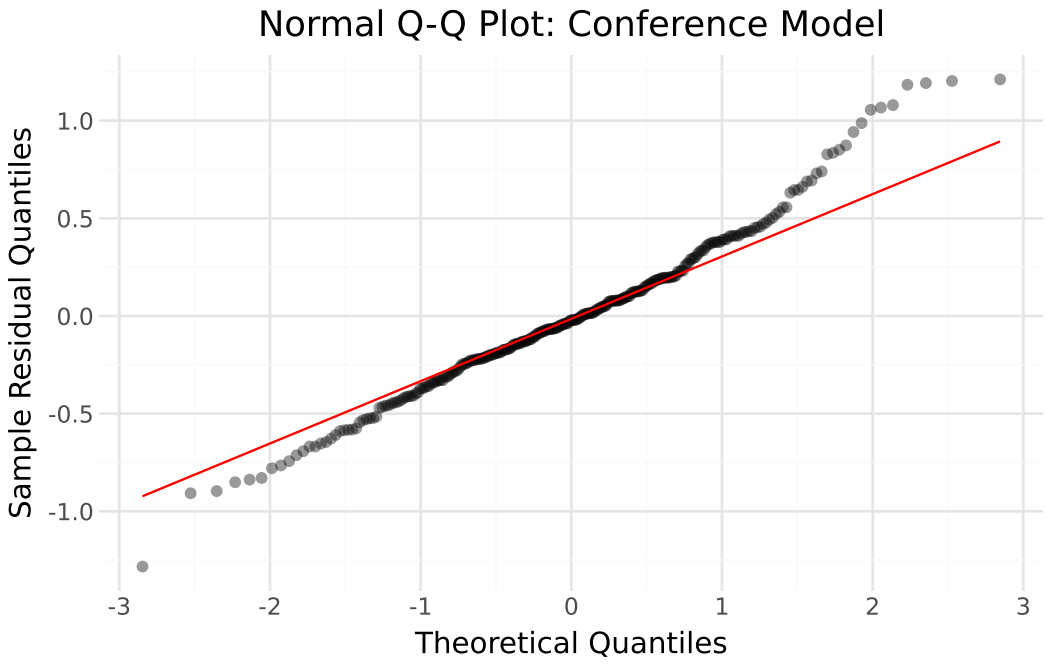
```
print("Breusch-Pagan p-value:", round(bp_pvalue, 4))
print("Breusch-Pagan F-test p-value:", round(bp_f_pvalue, 4))
```

```
Breusch-Pagan LM statistic: 55.917
Breusch-Pagan p-value: 0.0055
Breusch-Pagan F-test p-value: 0.0031
```

Normality

Q-Q Plot

```
(ggplot(cf_diag, aes(sample='resid_conf'))
    + stat_qq(alpha=0.4)
    + stat_qq_line(color='red')
    + labs(
        title='Normal Q-Q Plot: Conference Model',
        x='Theoretical Quantiles',
        y='Sample Residual Quantiles'
    )
    + theme_minimal()
)
```



Our Q-Q Plot points close to the reference line which suggests normality. We have a little tail in the end which could be a problem, but we should not automatically remove those teams

Identifiability

Check design-matrix rank

```
# Compare design-matrix rank with the number of columns
design_matrix = fit_conf.model.exog

matrix_rank = np.linalg.matrix_rank(design_matrix)
n_parameters = design_matrix.shape[1]

print("Design-matrix rank:", matrix_rank)
print("Number of model parameters:", n_parameters)
print("Full column rank:", matrix_rank == n_parameters)
```

```
Design-matrix rank: 33
Number of model parameters: 33
Full column rank: True
```

Calculate Variance Inflation Factors

```
# Construct numerical predictor matrix including an intercept
vif_design = sm.add_constant(
    cf_train.select(['bpr'] + conf_cols).to_pandas(),
    has_constant='add'
).astype(float)

# Compute VIF for predictors, excluding the intercept
vif_table = pl.DataFrame({
    'predictor': list(vif_design.columns[1:]),
    'vif': [
        variance_inflation_factor(vif_design.values, i)
        for i in range(1, vif_design.shape[1])
    ]
}).sort('vif', descending=True)

print("Highest VIF values:")
```

```
print(vif_table.head(10))

print("Maximum VIF:", round(vif_table['vif'].max(), 3))
```

```
Highest VIF values:
shape: (10, 2)

 predictor                vif
 ---                      ---
 str                      f64

 bpr                      4.415567
 conference_MAAC          2.687855
 conference_SWAC          2.65073
 conference_SunBelt       2.263721
 conference_CAA           2.202082
 conference_AmericaEast   2.180744
 conference_Horizon       2.173925
 conference_MAC           2.165795
 conference_ASUN          2.150882
 conference_Southland     2.144534

Maximum VIF: 4.416
```

Model Refinements

Identify Influential Teams

```
# Calculate influence diagnostics
influence = fit_conf.get_influence()
dffits_values = influence.dffits[0]

# Conventional screening threshold
n = train_raw.height
p = len(fit_conf.params)

dffits_cutoff = 2 * np.sqrt(p / n)

# Attach influence measures to the training observations
influence_data = cf_diag.with_columns(
```

```
    pl.Series('dffits', dffits_values),
    pl.Series('abs_dffits', np.abs(dffits_values))
)

influential_teams = (
    influence_data
    .filter(pl.col('abs_dffits') > dffits_cutoff)
    .sort('abs_dffits', descending=True)
    .select(
        'team', 'conference', 'nil_m',
        'bpr', 'resid_conf', 'dffits'
    )
)

print("DFFITS threshold:", round(dffits_cutoff, 3))
print("Potentially influential teams:", influential_teams.height)
```

```
DFFITS threshold: 0.684
Potentially influential teams: 18
```

Fit

Ordinary Least Squares

We used OLS because it estimates regression coefficients by minimizing the sum of squared residuals between oberved and predicted vales.

Our model is:

$$\log(Y_i) = \beta_0 + \beta_1 BPR_i + \sum_{c \neq \text{BigTen}} \gamma_c I(C_i = c) + \epsilon_i$$

Where Y_i is estimated team NIL in millions of dollars, β_1 represents the association between BPR and log NIL, and γ_c represents the difference associated with conference membership relative to the Big Ten.

```
# Reuse the conference-adjusted training model
fr_fit = fit_conf

# Report key OLS model statistics
```

```
ols_summary = pl.DataFrame({
    'statistic': [
        'Training observations',
        'R-squared',
        'Adjusted R-squared',
        'Residual standard error',
        'Model F-test p-value'
    ],
    'value': [
        fr_fit.nobs,
        fr_fit.rsquared,
        fr_fit.rsquared_adj,
        np.sqrt(fr_fit.mse_resid),
        fr_fit.f_pvalue
    ]
})

print(
    ols_summary.with_columns(
        pl.col('value').round(4)
    )
)
```

```
shape: (5, 2)

 statistic                value
 ---                      ---
 str                      f64

 Training observations    282.0
 R-squared                0.8562
 Adjusted R-squared       0.8378
 Residual standard error  0.4339
 Model F-test p-value     0.0
```

Frequentist Inference

Frequentist treats the parameters as fixed but unknown quatites. We quantify estimation ucertainty using standard errors, 95% confience intervals, and hypothesis tests.

For each coefficient, we test the null hypothesis that the parameter equals zero. A coefficient is statistically significant at the 5% level when its corresponding two-sided p-value is below 0.05, assuming the model and assumptions are met

A 95% confidence interval is constructed using a procedure that would contain the true population parameter in approximately 95% of repeated samples under the model assumptions. It is not a direct 95% probability statement about the fixed parameter.

Since the response variable is log NIL, coefficients are converted to percentage differences on the original NIL scale:

$$100(e^{\hat{\beta}} - 1)$$

This transformation gives an easier way to interpreted our results

Calculate coefficient estimates and intervals

```
# Estimate standard 95% OLS confidence intervals
fr_ci = fr_fit.conf_int(alpha=0.05)

# Organize coefficient estimates and uncertainty
fr_table = (
    pl.DataFrame({
        'term': fr_fit.params.index.tolist(),
        'estimate': fr_fit.params.to_numpy(),
        'lower': fr_ci.iloc[:, 0].to_numpy(),
        'upper': fr_ci.iloc[:, 1].to_numpy(),
        'p_value': fr_fit.pvalues.to_numpy()
    })
    .with_columns(
        pct_estimate=(pl.col('estimate').exp() - 1) * 100,
        pct_lower=(pl.col('lower').exp() - 1) * 100,
        pct_upper=(pl.col('upper').exp() - 1) * 100,
        significant=pl.col('p_value') < 0.05
    )
)

# Display selected coefficients relevant to the business objective
selected_terms = [
    'bpr',
    'conference_BigEast',
```

```
    'conference_SEC',
    'conference_WestCoast'
]

(
    fr_table
    .filter(pl.col('term').is_in(selected_terms))
    .select(
        'term',
        'pct_estimate',
        'pct_lower',
        'pct_upper',
        'p_value',
        'significant'
    )
    .with_columns(pl.col(pl.Float64).round(3))
)
```

term str	pct_estimate f64	pct_lower f64	pct_upper f64	p_value f64	significant bool
”bpr”	2.867	1.804	3.941	0.0	true
”conference_BigEast”	17.137	-19.15	69.71	0.402	false
”conference_SEC”	10.347	-19.929	52.071	0.546	false
”conference_WestCoast”	-33.2	-54.969	-0.907	0.045	true

Bayesian Inference

Bayesian gives a different way for estimating regression paramters and quantifying uncertainty.

The theory is that you combine prior inormation with the likelihood of the observed data to get posterior parameter distributions

$$p(\theta \mid D) \propto p(D \mid \theta)p(\theta)$$

Following the course methodology, we specify diffuse uniform priors for the regression coefficients and an exponential prior for the residual standard deviation:

$$\beta_j \sim Uniform(-100, 100)$$

$$\sigma \sim Exponential(1)$$

These priors place relatively weak restrictions on the regression coefficients over the selected ranges. The uniform bounds are intentionally broad and are not based on substantive NIL market information. They nevertheless remain assumptions of the Bayesian model.

Bayesian Regression

```
import arviz as az
import pytensor

# Match the PyTensor configuration from Milestone 9
pytensor.config.cxx = ""

import bambi as bmb

# Reuse precisely the same predictors and observations as OLS
bayes_predictors = ['bpr'] + conf_cols

bayes_data = cf_train.select(
    ['log_nil'] + bayes_predictors
).to_pandas()

bayes_formula = (
    'log_nil ~ ' + ' + '.join(bayes_predictors)
)

# Specify the Gaussian regression model
ba_model = bmb.Model(
    bayes_formula,
    data=bayes_data,
    family='gaussian'
)

# Define the priors used in Milestone 9
priors_dict = {
    'Intercept': bmb.Prior(
        'Uniform', lower=-100, upper=100
    ),
    **{
```

```
        term: bmb.Prior(
            'Uniform', lower=-100, upper=100
        )
        for term in bayes_predictors
    },
    'sigma': bmb.Prior('Exponential', lam=1)
}

ba_model.set_priors(priors=priors_dict)

# Sample the posterior distributions
ba_fit = ba_model.fit(
    draws=2000,
    tune=2000,
    chains=4,
    cores=2,
    target_accept=0.90,
    random_seed=42
)
```

```
Initializing NUTS using jitter+adapt_diag...
/home/spencer/Workspace/data5600_project1/.venv/lib/python3.12/site-packages/pytensor/tensor/
Multiprocess sampling (4 chains in 2 jobs)
NUTS: [sigma, Intercept, bpr, conference_ACC, conference_ASUN, conference_AmericaEast, confe
```

```
Output()
```

```
/home/spencer/Workspace/data5600_project1/.venv/lib/python3.12/site-packages/pymc/step_metho
```

```
Sampling 4 chains for 2_000 tune and 2_000 draw iterations (8_000 + 8_000 draws total) took
```

Comparing both models

The frequentist and Bayesian models estimate the relationship between projected basketball performance and estimated NIL valuations while controlling for conference affiliation. Because both models use the same response, predictors, and training observations, their parameter estimates and uncertainty intervals can be compared directly.

The BPR coefficient represents the conditional association between a one-point difference in projected basketball strength and log NIL, holding conference fixed. Exponentiating the coefficient converts this relationship into a percentage difference in NIL valuations.

Conference coefficients represent differences relative to the Big Ten, holding projected BPR constant. These estimates describe conditional differences in NIL valuations across conferences rather than causal effects of conference membership.

Frequentist confidence intervals and Bayesian credible intervals have different interpretations. A 95% confidence interval refers to the long-run coverage of an estimation procedure, while a 95% credible interval represents posterior uncertainty about the parameter conditional on the specified prior and likelihood.

The resulting coefficient estimates will inform our interpretation of the NIL market, while their predictive usefulness will be assessed separately using held-out observations in the Evaluate stage.

Evaluate

Model Comparison

```
import pandas as pd

# Model 1: BPR alone, fitted on the original training sample
fit_bpr = smf.ols(
    'log_nil ~ bpr',
    data=train_raw.to_pandas()
).fit()

# Model 2: BPR + Conference, already fitted in Reconcile
fit_conference = fit_conf

# Prepare the held-out test observations
test_pd = test_raw.to_pandas()

# Check for unseen conference categories
unseen = (
    set(test_raw['conference'].to_list())
    - set(train_raw['conference'].to_list())
)
assert not unseen, f"Unseen test conferences: {unseen}"
```

```
# Match the test dummy columns to the training specification
test_dummies = pd.get_dummies(
    test_pd['conference'],
    prefix='conference',
    dtype=int
).reindex(columns=conf_cols, fill_value=0)

test_design = pd.concat(
    [
        test_pd[['bpr']].reset_index(drop=True),
        test_dummies.reset_index(drop=True)
    ],
    axis=1
)

# Generate predictions on the log scale
pred_bpr_log = fit_bpr.predict(test_pd)
pred_conf_log = fit_conference.predict(test_design)

# Apply smearing corrections estimated using training residuals
smear_bpr = np.mean(np.exp(fit_bpr.resid))
smear_conf = np.mean(np.exp(fit_conference.resid))

pred_bpr = np.exp(pred_bpr_log) * smear_bpr
pred_conf = np.exp(pred_conf_log) * smear_conf

# Evaluate predictions against observed NIL estimates ($ millions)
actual_nil = test_pd['nil_m'].to_numpy()

def evaluate_predictions(name, prediction):
    errors = actual_nil - np.asarray(prediction)
    mse = np.mean(errors ** 2)
    return {
        'model': name,
        'MSE ($M squared)': mse,
        'RMSE ($M)': np.sqrt(mse),
        'MAE ($M)': np.mean(np.abs(errors))
    }

model_results = pl.DataFrame([
    evaluate_predictions('BPR only', pred_bpr),
    evaluate_predictions('BPR + Conference', pred_conf)
```

```
])

print(model_results.with_columns(
    pl.col(pl.Float64).round(3)
))
```

```
shape: (2, 4)

 model              MSE ($M squared)   RMSE ($M)   MAE ($M)
 ---                ---                ---         ---
 str                f64                f64         f64

 BPR only           2.832              1.683       0.981
 BPR + Conference   0.886              0.941       0.598
```

The two regression models were evaluated using 71 held-out observations, representing 20% of the analytical dataset. Both models predict estimated NIL valuations from projected BPR, but the second model additionally controls for conference membership. Predictions were converted from log NIL to estimated NIL in millions of dollars using a smearing adjustment based on the training residuals.

The BPR-only model produced a test MSE of 2.832 ($M squared), an RMSE of $1.683 million, and an MAE of 0.981$million.Incomparison, theBPR+Conferencemodelachievedasubstantiallylower MSEof0$ squared), an RMSE of $0.941 million, and an MAE of $0.598 million.

Including conference affiliation reduced test MSE by approximately 68.7%, RMSE by 44.1%, and MAE by 39.0% relative to the BPR-only model. These improvements indicate that conference information provides meaningful predictive value beyond projected basketball performance alone.

We therefore select the BPR + Conference model as our preferred specification because it minimizes held-out squared prediction loss, consistent with the business objective established in the Plan section.

Predict

After using our conference-adjusted model, we refit the model using the 353 matched teams to get the final NIL benchmarks

For each team, we predict log NIL using BPR and conference. We then convert the prediction to millioons of dollars and apply a residual smearing adjustment to estimate the conditional mean NIL

Team NIL Benchmarks

```
# Refit the selected specification on all matched teams
final_data = tm_data.sort('team')
final_pd = final_data.to_pandas()

# Encode conference; Big Ten remains the reference
full_design = final_pd[['bpr', 'log_nil']].copy()

full_dummies = pd.get_dummies(
    final_pd['conference'],
    prefix='conference',
    dtype=int
)

assert 'conference_BigTen' in full_dummies.columns

full_conf_cols = sorted(
    c for c in full_dummies.columns
    if c != 'conference_BigTen'
)

full_design = pd.concat(
    [full_design, full_dummies[full_conf_cols]],
    axis=1
)

# Fit the final conference-adjusted OLS model
final_fit = smf.ols(
    'log_nil ~ bpr + ' + ' + '.join(full_conf_cols),
    data=full_design
).fit()

# Model predictions on the log NIL scale
pred_log = np.asarray(final_fit.predict(full_design))

# Convert to median and smearing-adjusted mean NIL
smearing_factor = np.mean(np.exp(final_fit.resid))

pred_median = np.exp(pred_log)
pred_mean = pred_median * smearing_factor
```

```
# Create the final benchmark dataset
benchmarks = (
    final_data
    .with_columns(
        pl.Series('median_benchmark_m', pred_median),
        pl.Series('benchmark_nil_m', pred_mean)
    )
    .with_columns(
        gap_m=pl.col('benchmark_nil_m') - pl.col('nil_m')
    )
    .with_columns(
        bpr_rank=pl.col('bpr')
            .rank('min', descending=True)
            .over('conference')
            .cast(pl.Int64),
        gap_rank=pl.col('gap_m')
            .rank('min', descending=True)
            .over('conference')
            .cast(pl.Int64),
        conference_teams=pl.len().over('conference')
    )
)

print('Teams benchmarked:', benchmarks.height)
print('Final model R-squared:', round(final_fit.rsquared, 3))
print('Smearing factor:', round(smearing_factor, 3))
```

```
Teams benchmarked: 353
Final model R-squared: 0.858
Smearing factor: 1.092
```

Identifying Teams Above and Below Benchmark

```
# Largest positive gaps: estimated NIL below benchmark
top_below = (
    benchmarks
    .filter(pl.col('gap_m') > 0)
    .sort('gap_m', descending=True)
    .select(
        'team', 'conference', 'bpr',
```

```
        'nil_m', 'benchmark_nil_m', 'gap_m'
    )
    .head(8)
)

# Largest negative gaps: estimated NIL above benchmark
top_above = (
    benchmarks
    .filter(pl.col('gap_m') < 0)
    .sort('gap_m')
    .select(
        'team', 'conference', 'bpr',
        'nil_m', 'benchmark_nil_m', 'gap_m'
    )
    .head(8)
)

print("TEAMS BELOW THEIR NIL BENCHMARK")
print(
    top_below.with_columns(
        pl.col(pl.Float64).round(2)
    )
)

print("\nTEAMS ABOVE THEIR NIL BENCHMARK")
print(
    top_above.with_columns(
        pl.col(pl.Float64).round(2)
    )
)
```

```
TEAMS BELOW THEIR NIL BENCHMARK
shape: (8, 6)

 team            conference   bpr     nil_m   benchmark_nil_m   gap_m
 ---             ---          ---     ---     ---               ---
 str             str          f64     f64     f64               f64

 Butler          BigEast      6.18    2.35    6.57              4.22
 Georgetown      BigEast      10.65   3.35    7.36              4.01
 SMU             ACC          11.3    3.35    7.32              3.97
 VCU             Atlantic10   14.13   2.4     6.13              3.73
```

```
 Nebraska         BigTen       18.36   4.6     8.23              3.63
 Auburn           SEC          18.49   5.0     8.38              3.38
 Clemson          ACC          13.49   4.5     7.74              3.24
 South Carolina   SEC          5.45    2.95    6.02              3.07


TEAMS ABOVE THEIR NIL BENCHMARK
shape: (8, 6)

 team           conference   bpr     nil_m   benchmark_nil_m   gap_m
 ---            ---          ---     ---     ---               ---
 str            str          f64     f64     f64               f64

 Kentucky       SEC          19.46   15.8    8.59              -7.21
 Indiana        BigTen       13.32   14.1    7.24              -6.86
 Gonzaga        Pac12        25.03   12.1    5.45              -6.65
 Memphis        American     5.78    9.2     3.06              -6.14
 Saint Mary's   WestCoast    4.98    9.35    3.44              -5.91
 Seton Hall     BigEast      7.8     11.5    6.84              -4.66
 Louisville     ACC          22.23   13.9    9.66              -4.24
 Miami (Fla.)   ACC          22.69   13.5    9.77              -3.73
```

Utah State Case Study

Utah State University provides a practical case study for interpreting the model's NIL benchmarks. We compare Utah State's estimated NIL valuation with the predicted value associated with its projected BPR and conference membership.

We also compare Utah State with other teams in its conference to assess its relative projected basketball strength and benchmark gap. These comparisons identify potential funding questions for further investigation, not a definitive recommendation to increase NIL expenditures.

```python
# Locate Utah State
usu = benchmarks.filter(
    pl.col('team') == 'Utah State'
)

assert usu.height == 1, "Check Utah State team name"

# Extract case-study values
usu_conference = usu['conference'][0]
```

```
usu_nil = float(usu['nil_m'][0])
usu_bpr = float(usu['bpr'][0])
usu_median = float(usu['median_benchmark_m'][0])
usu_benchmark = float(usu['benchmark_nil_m'][0])
usu_gap = float(usu['gap_m'][0])

print("UTAH STATE NIL CASE STUDY")
print("Conference:", usu_conference)
print("Projected BPR:", round(usu_bpr, 2))
print(f"Estimated NIL: ${usu_nil:.2f}M")
print(f"Median benchmark: ${usu_median:.2f}M")
print(f"Mean NIL benchmark: ${usu_benchmark:.2f}M")
print(f"Benchmark gap: ${usu_gap:+.2f}M")

print(
    "BPR rank in conference:",
    int(usu['bpr_rank'][0]),
    "of",
    int(usu['conference_teams'][0])
)

print(
    "Gap rank in conference:",
    int(usu['gap_rank'][0]),
    "of",
    int(usu['conference_teams'][0])
)

# Show all of Utah State's conference peers
usu_peers = (
    benchmarks
    .filter(pl.col('conference') == usu_conference)
    .sort('gap_m', descending=True)
    .select(
        'team', 'bpr', 'nil_m',
        'benchmark_nil_m', 'gap_m'
    )
)

print("\nCONFERENCE COMPARISON")
print(
    usu_peers.with_columns(
```

```
        pl.col(pl.Float64).round(2)
    )
)
```

```
UTAH STATE NIL CASE STUDY
Conference: Pac12
Projected BPR: 13.73
Estimated NIL: $2.00M
Median benchmark: $3.75M
Mean NIL benchmark: $4.10M
Benchmark gap: $+2.10M
BPR rank in conference: 2 of 9
Gap rank in conference: 1 of 9

CONFERENCE COMPARISON
shape: (9, 5)

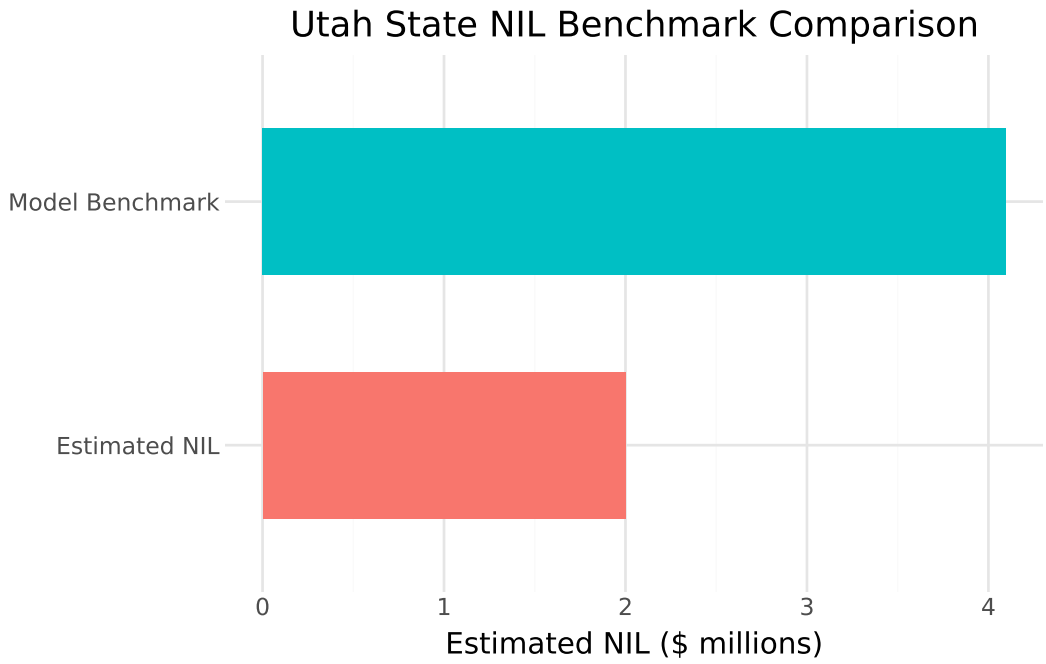
 team              bpr    nil_m  benchmark_nil_m  gap_m
 ---               ---    ---    ---              ---
 str               f64    f64    f64              f64

 Utah State        13.73  2.0    4.1              2.1
 Texas State       1.28   1.5    2.99             1.49
 Colorado State    8.86   2.5    3.62             1.12
 Oregon State      7.94   3.0    3.54             0.54
 Washington State  -1.72  2.5    2.77             0.27
 Boise State       7.88   3.5    3.53             0.03
 San Diego State   8.26   4.0    3.57             -0.43
 Fresno State      -0.75  4.0    2.84             -1.16
 Gonzaga           25.03  12.1   5.45             -6.65
```

```
usu_plot = pl.DataFrame({
    'category': ['Estimated NIL', 'Model Benchmark'],
    'nil_m': [usu_nil, usu_benchmark]
})

(ggplot(usu_plot, aes(x='category', y='nil_m', fill='category'))
    + geom_col(width=0.6)
    + coord_flip()
    + labs(
```

```
        title='Utah State NIL Benchmark Comparison',
        x='',
        y='Estimated NIL ($ millions)'
    )
    + theme_minimal()
    + theme(legend_position='none')
)
```



Communicate

Business Recommendations

The analysis suggests that conference-adjusted NIL benchmarks provide a more useful basis for comparing programs than projected basketball performance alone.

First, athletic departments and NIL collectives should benchmark their estimated financial resources against programs with comparable projected basketball strength and conference membership. The significant improvement in test prediction accuracy indicates that conference context helps explain variation in NIL valuations beyond BPR alone.

Second, programs with large positive benchmark gaps should be prioritized for further financial and roster analysis. A positive gap indicates that a team's estimated NIL valuation is below

the amount predicted by the model. These differences may suggest opportunities to investigate funding, recruiting, and player retention strategies, although they do not independently establish underfunding.

Third, programs with negative benchmark gaps should evaluate whether their higher estimated NIL valuations align with their strategic priorities.Higher-than-benchmark valuations may reflect recruiting investments, institutional resources, or marketability that the model does not directly measure. They should not automatically be interpreted as inefficient spending.

Finally, Utah State provides a practical application of this framework. Comparing its projected BPR, NIL estimate, conference standing, and model-implied benchmark helps identify whether its estimated resources are relatively high or low for its projected basketball strength. Before recommending any change in funding, Utah State would need verified NIL commitments, roster-retention information, and a clearer understanding of the expected return on additional spending.

Limitations and Uncertainty

Although the selected regression model provides useful benchmarks, several limitations affect the conclusions.

First, estimated NIL valuations are not verified team expenditures, and preseason BPR measures projected rather than realized basketball performance. Consequently, the model evaluates associations between estimates rather than the financial returns from actual NIL investments.

Second, the analysis uses cross-sectional data from a single season. It cannot determine whether increasing NIL expenditures leads to stronger future performance, nor can it fully capture factors such as recruiting reputation, donor contributions, athletic revenue, and media exposure.

Third, the model assumes a common BPR slope across conferences and relies on a finite sample of teams. Predictions and team rankings are subject to model uncertainty, measurement error, and possible omitted-variable bias.

Future research could incorporate verified NIL payments, actual season outcomes, player-level performance, athletic department revenues, and multiple seasons of data. These additions would help distinguish market valuations from actual expenditures and support more meaningful analysis of the relationship between NIL investment and basketball performance.

Conclusion

This project developed a regression-based framework for evaluating estimated NIL valuations across Division I men's basketball programs. The analysis found a positive association between

projected basketball strength and NIL valuations, while accounting for conference affiliation substantially improved predictive performance.

The selected conference-adjusted model achieved a held-out RMSE of $0.941 million and was used to create team-level NIL benchmarks. These benchmarks provide a consistent method for comparing programs and identifying those whose estimated valuations differ from model expectations.

Ultimately, the analysis demonstrates how regression and statistical inference can support financial benchmarking in college athletics. While the results cannot establish which programs are definitively overfunded or underfunded, they provide an evidence-based starting point for more informed NIL planning and resource-allocation discussions.